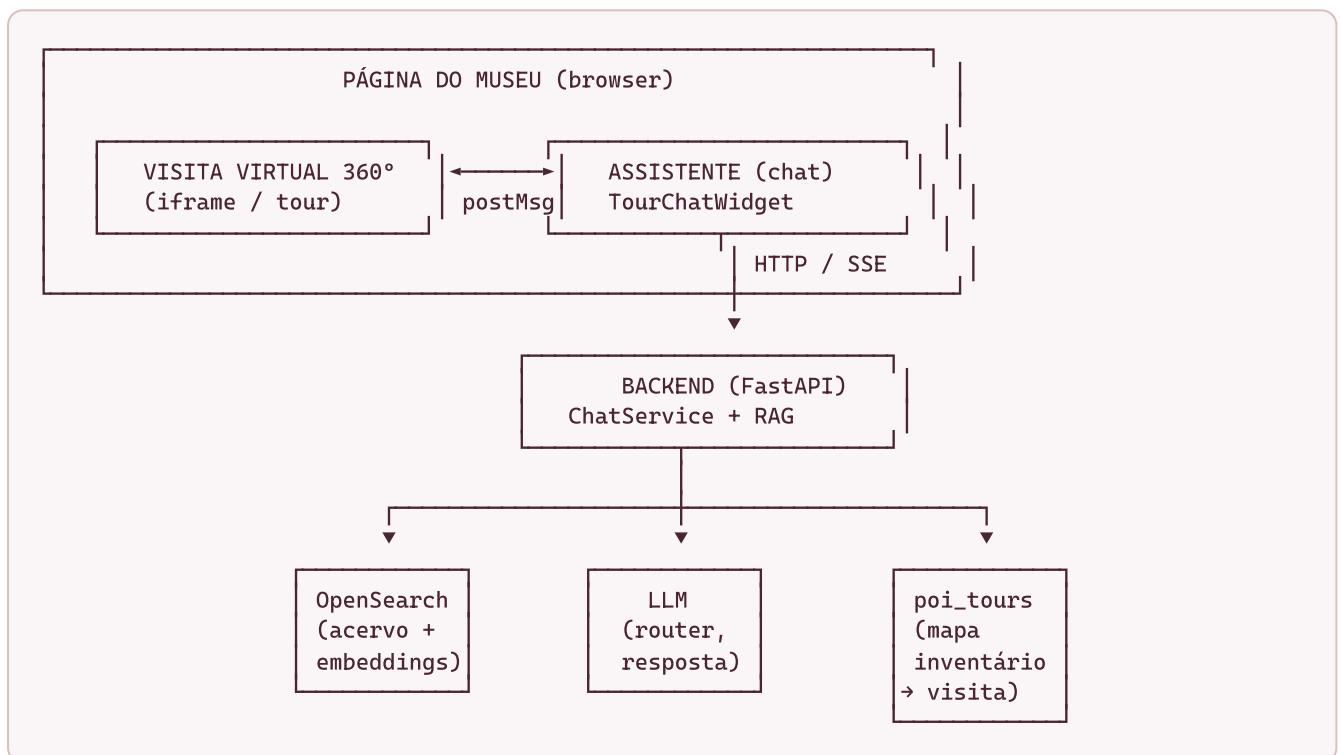


Património 360 — Assistente Virtual de Museus

Documento de apresentação **estrutural** (não técnico-exaustivo). Cobre três temas: **(1)** a pipeline RAG e os seus caminhos possíveis, **(2)** a comunicação Frontend ↔ Backend, **(3)** a comunicação Frontend ↔ Visita Virtual.

1. Visão geral em uma frase

O projeto é um **assistente conversacional embebido numa visita virtual 360° de um museu**. O visitante faz perguntas (por texto, imagem ou modelo 3D), o sistema procura nas peças do acervo do museu e responde em linguagem natural — e, quando faz sentido, **leva o visitante até ao ponto exato da visita virtual** onde a peça está exposta.



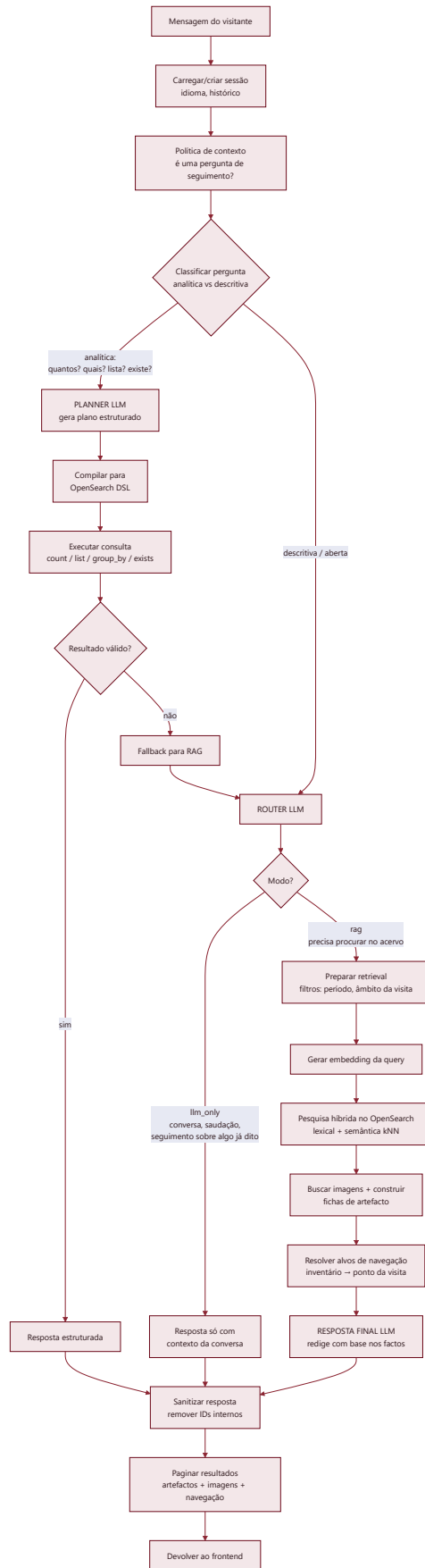
Peças-chave:

- **Frontend (React)** — widget de chat sobreposto à visita virtual.
- **Backend (FastAPI / ChatService)** — orquestra todo o raciocínio.
- **OpenSearch** — base de pesquisa do acervo (texto + vetores semânticos + imagens).
- **LLM** — usado em três momentos: decidir o caminho (*router*), planejar consultas analíticas (*planner*) e redigir a resposta final.
- **poi_tours/** — ficheiros JSON que ligam o **número de inventário** de uma peça ao **ponto da visita virtual** (panorama + overlay).

2. A Pipeline RAG e os seus caminhos

RAG = *Retrieval-Augmented Generation*: o modelo **não responde de cor** — primeiro **procura** factos no acervo do museu e só depois **redige** a resposta com base nesses factos.

A grande característica deste projeto é que **não existe um único caminho**. Consoante a pergunta, o sistema escolhe entre vários percursos. O diagrama abaixo mostra-os todos.

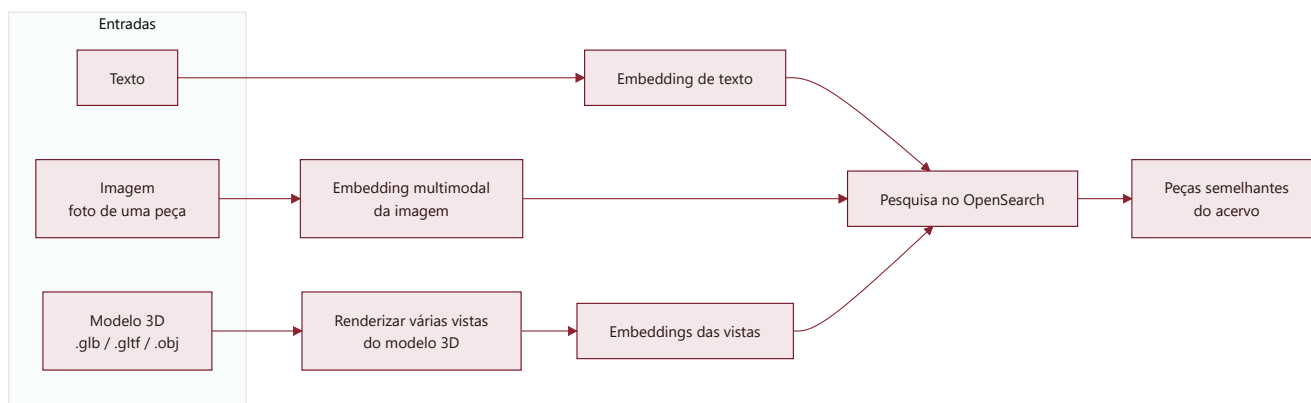


2.1. Os caminhos, explicados

Caminho	Quando é escolhido	Exemplo de pergunta	O que acontece
Estruturado	Perguntas analíticas/contáveis	"Quantos azulejos do séc. XVIII há no museu?", "Lista os retratos"	O LLM planeia uma consulta exata (contagem, lista, agrupamento) que é compilada para uma query OpenSearch e executada — números fiáveis, não "inventados".
RAG	Perguntas abertas/descriptivas que precisam de factos do acervo	"Fala-me sobre o traje de criança", "Que peças há sobre arte sacra?"	Procura semântica + lexical no acervo, junta as peças mais relevantes e o LLM redige a resposta só com base nelas .
LLM-only	Conversa , saudações, ou seguimento sobre algo já dito	"Olá", "E quem foi o autor dessa?"	Não vai à base de dados; responde com o contexto da própria conversa.
Fallback	O caminho estruturado falhou ou teve baixa confiança	—	Reverte automaticamente para o caminho RAG, garantindo sempre uma resposta.

2.2. Entradas multimodais (variações do caminho RAG)

O mesmo motor RAG aceita três tipos de "pergunta":



- **Texto** → pesquisa híbrida (palavras + significado).
- **Imagem** → o visitante envia uma foto; geramos um *embedding* multimodal e procuramos as peças **visualmente parecidas**.
- **Modelo 3D** → renderizamos várias vistas do objeto, geramos *embeddings* de cada uma e procuramos as peças mais semelhantes.

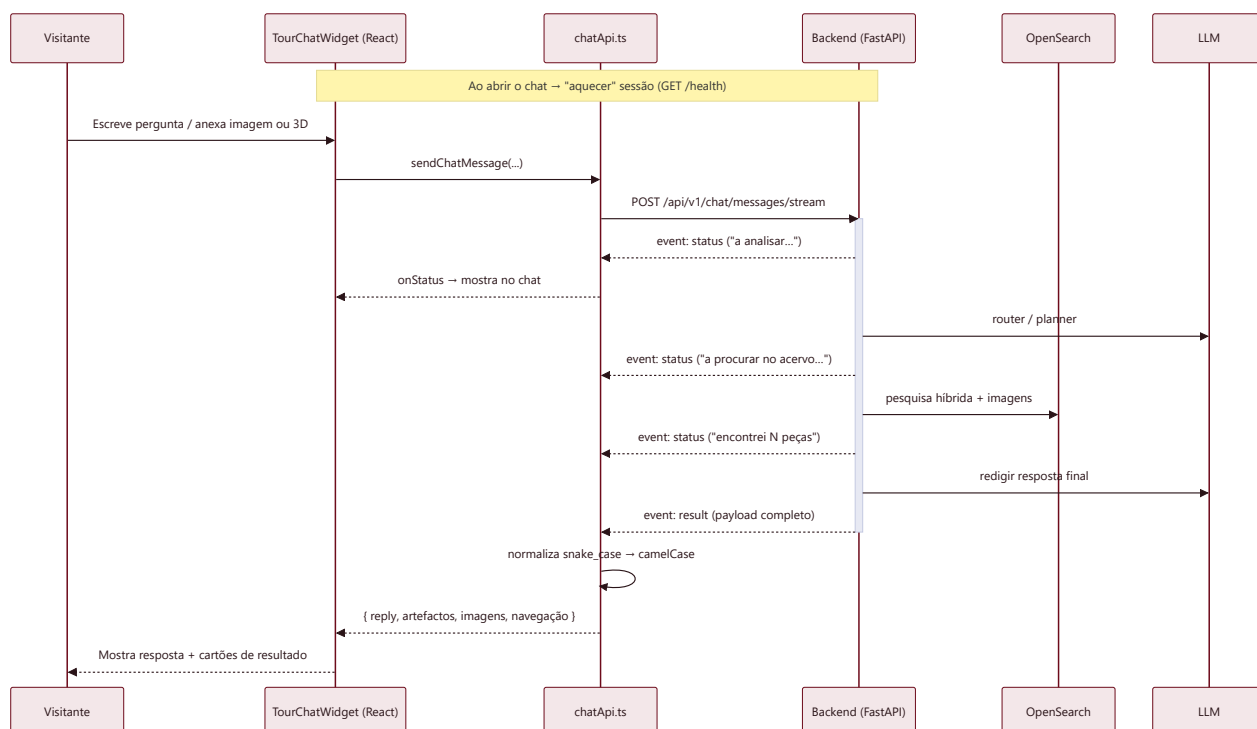
2.3. Onde entra cada componente externo

- **LLM** é chamado em até **3 momentos** por pergunta: *router* (decidir o caminho), *planner* (só no caminho estruturado) e *resposta final*.

- **OpenSearch** guarda o acervo com três "vistas" do mesmo dado: texto pesquisável, vetores semânticos (texto) e vetores de imagem.
- **poi_tours/** é consultado **no fim**, para transformar o nº de inventário das peças encontradas em **pontos navegáveis da visita virtual**.

3. Comunicação Frontend ↔ Backend

A comunicação é **HTTP REST**, com uma camada de **streaming (SSE)** para mostrar o progresso ao vivo ("a analisar pedido...", "a procurar no acervo...", "encontrei N peças...").



3.1. Endpoints principais (todos sob `/api/v1/chat`)

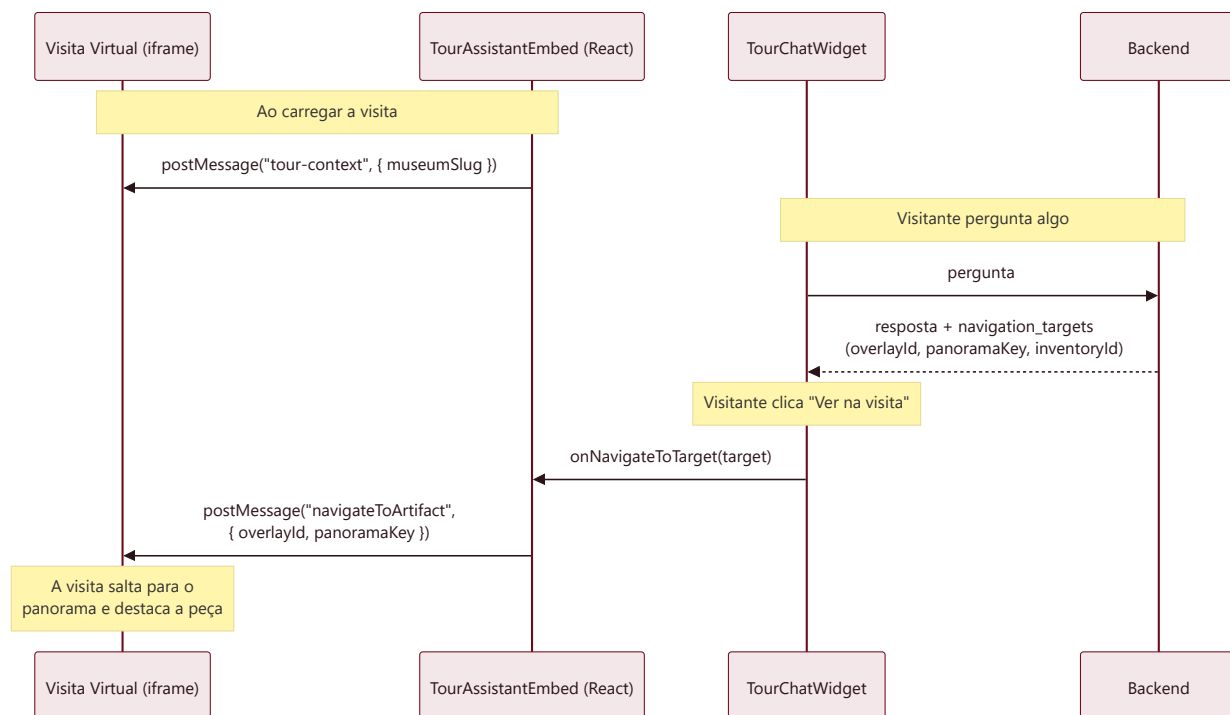
Endpoint	Função
<code>GET /health</code>	"Aquecer" a sessão e confirmar que o backend está vivo.
<code>POST /messages</code> e <code>/messages/stream</code>	Pergunta por texto (versão simples e versão com progresso ao vivo).
<code>POST /messages/image</code> e <code>/image/stream</code>	Pergunta com imagem anexada (upload multipart).
<code>POST /messages/model</code> e <code>/model/stream</code>	Pergunta com modelo 3D anexado.
<code>POST /messages/regenerate</code>	Voltar a gerar a última resposta.
<code>POST /messages/results</code>	Paginação — "ver mais resultados".
<code>GET /artifacts/{id}/detail-context</code>	Contexto relacional de uma peça (autores, conjuntos, exposições) — carregado só quando o visitante abre o modal .
<code>GET /artifacts/related</code>	Paginação dos artefactos relacionados (dentro do modal).
<code>GET /artifacts/{id}/full</code>	Ficha completa de uma peça (com todas as imagens).
<code>GET /images/{ref}</code>	Servir os ficheiros de imagem das peças.

3.2. Princípios de desenho

- **Estado de sessão no backend** — o frontend só guarda o `conversationId` . O histórico, os filtros ativos e os últimos resultados ficam do lado do servidor (permite seguimento: "e a anterior?").
- **Streaming opcional** — se o frontend passa um `callback` de progresso, usa a versão `/stream` (SSE) e mostra mensagens de estado em tempo real; caso contrário, faz um pedido simples.
- **Tradução de formato** — o backend responde em `snake_case` ; o `chatApi.ts` normaliza tudo para `camelCase` e tipos do frontend, isolando a UI de mudanças no backend.
- **Carregamento preguiçoso (lazy)** — detalhes pesados (contexto relacional, ficha completa, páginas extra) só são pedidos quando o visitante interage.

4. Comunicação Frontend ↔ Visita Virtual

A visita virtual é uma aplicação **externa** carregada num `<iframe>` . O assistente e a visita comunicam por `postMessage` (mensagens entre janelas do browser) — não há servidor pelo meio.



4.1. As duas mensagens trocadas

Mensagem	Direção	Conteúdo	Quando
<code>patrimonio360:tour-context</code>	Assistente → Visita	<code>{ museumSlug }</code>	Quando a visita acaba de carregar (sincroniza o contexto do museu).
<code>navigateToArtifact</code>	Assistente → Visita	<code>{ overlayId, panoramaKey }</code>	Quando o visitante clica em "Ver na visita" num resultado.

4.2. Como nasce um "alvo de navegação"

Este é o elo que torna o projeto distinto — ligar **uma peça do acervo** a **um ponto físico da visita 360°**:

```

Peça encontrada no acervo
  |
  | (tem um número de inventário, ex. "MNAZ 1234")
  v
Backend consulta poi_tours/panorama-overlays-inventory-<museu>.json
  |
  | (índice: nº de inventário → overlay + panorama)
  v
navigation_target = { overlayId, panoramaKey, inventoryId, título, localização }
  |
  v
Frontend mostra botão "Ver na visita" → postMessage → a visita 360° salta para lá
  
```

- O serviço de navegação **normaliza** os números de inventário (maiúsculas, sem pontuação, lida com listas e variantes) para casar de forma robusta.
- Se uma peça **não estiver mapeada** na visita, simplesmente não aparece o botão de navegação — a resposta de texto continua a funcionar.

5. Resumo para a apresentação (1 slide)

- **Problema:** dar a uma visita virtual de museu um guia inteligente que percebe perguntas em linguagem natural e mostra fisicamente onde estão as peças.
- **Como:** assistente de chat embebido + pipeline **RAG multimodal** (texto, imagem, 3D) + ligação peça ↔ ponto da visita.
- **Inteligência adaptativa:** a pergunta é **encaminhada** para o caminho certo — *estruturado* (números fiáveis), *RAG* (respostas com factos do acervo) ou *conversa*.
- **Arquitetura limpa:** o frontend é "burro" (UI + ponte para a visita); todo o raciocínio e estado vivem no backend; OpenSearch é o cérebro de pesquisa; o LLM decide e redige.
- **Diferenciador:** o passo *inventário* → *ponto da visita 360°*, que transforma uma resposta de texto numa **experiência espacial guiada**.

